

COSC 39: Theory of Computation

Problem 1. Bogus proof for $P = NP$.

A boolean formula is in disjunctive normal form (or DNF) if it consists of a disjunction (OR) of several terms, each of which is the conjunction (AND) of one or more literals. For example, the formula

$$(\bar{x} \wedge y \wedge z) \vee (y \wedge z) \vee (x \wedge \bar{y} \wedge \bar{z})$$

is in disjunctive normal form.

Consider the satisfiability problem for DNFs:

DNFSAT

- **Input:** A boolean formula φ in disjunctive normal form
- **Output:** Is the formula φ satisfiable?

- (a) Describe a polynomial-time algorithm to solve DNFSAT.
- (b) What is the error in the following argument that $P = NP$?

We reduce the NP-hard problem 3SAT to DNFSAT. Suppose we are given a boolean formula in conjunctive normal form with at most three literals per clause, and we want to know if it is satisfiable. We can use the distributive law to construct an equivalent formula in disjunctive normal form. For example,

$$(x \vee y \vee \bar{z}) \wedge (\bar{x} \vee \bar{y}) = (x \wedge \bar{x}) \vee (y \wedge \bar{x}) \vee (\bar{z} \wedge \bar{x}) \vee (x \wedge \bar{y}) \vee (y \wedge \bar{y}) \vee (\bar{z} \wedge \bar{y})$$

Now we can use the algorithm from part (a) to determine, in polynomial time, whether the resulting DNF formula is satisfiable. We have just solved 3SAT in polynomial time. Since 3SAT is NP-hard, we must conclude that $P = NP$.

Solution.

- (a) A DNF is satisfiable if at least one of the clauses is satisfiable, and a clause is satisfiable

if and only if it doesn't contain a complementary pair (x and $\bar{x}$). The following algorithm checks this condition in $O(n)$ time where n is the total number of literals since `hasComplementaryPair` runs in $O(|C|)$ per clause (as each literal is visited at most once).

```
DNFSAT( $\varphi$ ):
for each clause  $C$  in  $\varphi$ :
    if NOT (has_complementary_pair( $C$ )):
        return "SATISFIABLE"
    return "UNSATISFIABLE"

has_complementary_pair( $C$ ):
    Input: A clause  $C$  (list of literals)
    Output: TRUE if  $C$  contains both  $x$  and  $\bar{x}$  for any variable  $x$ 

    seen  $\leftarrow$  empty set
    for each literal  $L$  in  $C$ :
        if complement( $L$ )  $\in$  seen:
            return TRUE
        add  $L$  to seen
    return FALSE
```

- (b) The proof converts 3SAT to DNFSAT. The error in the argument is that the reduction cannot be achieved in polynomial time. Let ϕ be CNF with m clauses and at most 3 literals. Then converting ϕ to its DNF form ψ requires picking one literal from each clause, which can produce up to 3^m terms, or $3^m \cdot m$ literals in the resulting DNF. Since the input size is $O(3m)$ and we get output size $O(3^m \cdot m)$, which is exponential in m , a polynomial time algorithm won't even be able to finish writing the clauses and hence this step cannot be performed in polynomial time, and the reduction fails.

Problem 2. Seer with predictive power.

We know that CIRCUITSAT is self-reducible. That is, if there is an oracle that decides whether a given circuit is satisfiable, then we can find the satisfying assignment in polynomial time.

Not surprisingly, oracles probably do not exist in real life. One day, a seer visits your door and claims that, while they have no way to decide if a given circuit is satisfiable or not, they hold the power to decide between two given circuits, which one is more likely to have a satisfying assignment.

In other words, if you hand the seer two circuits C and C' :

- the seer will correctly point out which circuit is satisfiable if exactly one of C and C' is satisfiable;
- the seer will choose an arbitrary circuit if both or neither C and C' are satisfiable.

Most importantly, the seer themselves do not know which case it is; since neither you nor them can check satisfiability, it is impossible to tell if the circuit the seer pointed out has a satisfying assignment or not.

Prove that you can find a satisfying assignment for any given circuit in polynomial time, with the help of the seer.

Solution.

We will prove the above seer can be used to find a satisfying assignment for any given circuit C in polynomial time.

Let C be a circuit that takes some assignment $x = (x_1, x_2, \dots, x_n)$ and outputs True or False depending on whether x satisfies C .

To find some assignment $(x_1, x_2, \dots, x_n)$ that satisfies C , we use the following algorithm. Note: we do not know whether C is satisfiable. We prove that if C is satisfiable, this algorithm will produce a satisfying assignment, and if C is unsatisfiable, it will produce an assignment that doesn't satisfy. The last step plugs the assignment into C to check whether C is satisfiable or not. (Let $C_0 = C$).

1. For $i = 1$ to n , do the following:

- (a) Consider the switch x_i . The value of x_i in a satisfying assignment must be either 0 or 1.
- (b) Construct two circuits C_i^1 and C_i^0 , where C_i^1 is C_{i-1} with x_i hardcoded to 1, and C_i^0 is C_{i-1} with x_i hardcoded to 0. Pass both into the seer. The following cases are possible:
 - C_{i-1} is satisfiable:

1. All satisfying assignments have $x_i = 1$. Then C_i^0 is unsatisfiable and C_i^1 is satisfiable. Thus, the seer returns C_i^1 .
2. All satisfying assignments have $x_i = 0$. Then C_i^1 is unsatisfiable and C_i^0 is satisfiable. Thus, The seer returns C_i^0 .
3. Both values appear in satisfying assignments. Then both C_i^1 and C_i^0 are satisfiable. Thus, the seer returns one arbitrarily.

****Note**** — In all three cases, the circuit C_i returned by the seer is satisfiable.

- C_{i-1} is unsatisfiable.
 1. Both C_i^1 and C_i^0 are unsatisfiable. The seer returns one arbitrarily, and C_i is unsatisfiable.

(c) Set C_i to whichever of C_i^0 or C_i^1 the seer returned.

Invariant for the loop: For every iteration, the invariants holds that C_i is satisfiable if and only if C_{i-1} is satisfiable, for all i .

Base Case: when $i = 0$, $C_0 = C$ is satisfiable iff C is satisfiable. Thus, the base case is true.

Inductive Step: Assuming C_{i-1} is satisfiable iff C is satisfiable, the cases in our loop shows that C_i is satisfiable iff C_{i-1} is satisfiable. Thus, our invariant holds.

2. **Loop Finished:** After n iterations, we have a circuit C_n in which all variables $x_1, \dots, x_n$ are hardcoded. By our invariant, C_n is satisfiable if and only if $C_0 = C$ is satisfiable. The hardcoded values $(x_1, \dots, x_n)$ are our final assignment (still unknown whether its satisfies C or not). To verify run the original circuit C on the derived assignment. If C outputs *True*, we have found a satisfying assignment. If C outputs *False*, then by our invariant, no satisfying assignment exists and C is unsatisfiable.

Since the seer is called exactly n times, the entire algorithm runs in polynomial time.